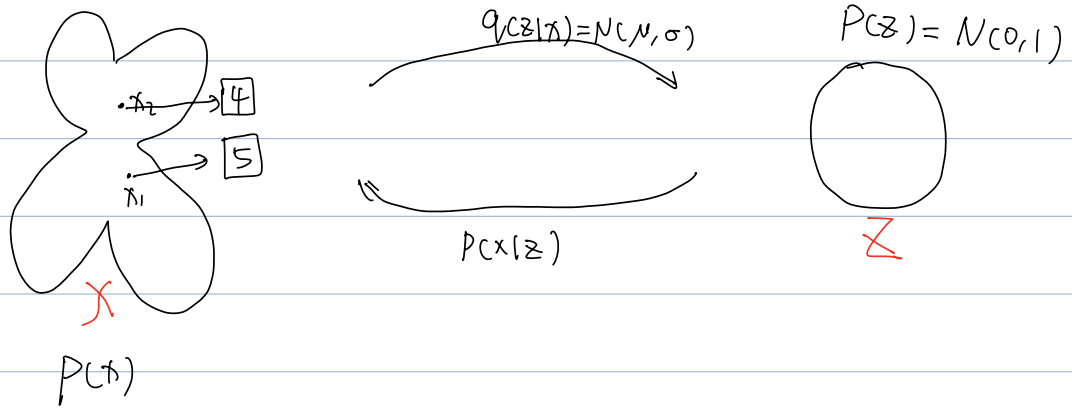
核心：对潜空间进行约束



由于 $p(x)$ 是一个概率分布，概率最大的点 x，其 $\log p(x)$ 值也一定最大

这里以 $\frac{1}{\sqrt{2\pi}}e^{-\frac{x^2}{2}}$ 正态分布为例，$\log p(x) = -\frac{x^2}{2} - \ln\sqrt{2\pi}$，在 0 处取到最大值，符合预期

因此计算：$\max \log p(x)$

由于一个 x 可能对应很多 z：

$$\log p(x) = \log \int p(x,z)\,dz$$

$p(x,z)$ 定义域为笛卡尔积空间 $x \times z$

$$= \log \int \frac{q(z|x)}{q(z|x)} p(x,z)\,dz$$

$$= \log E_{q(z|x)\sim z}\left[\frac{p(x,z)}{q(z|x)}\right]$$

$$\geq E_{q(z|x)\sim z} \log\left(\frac{p(x,z)}{q(z|x)}\right)$$

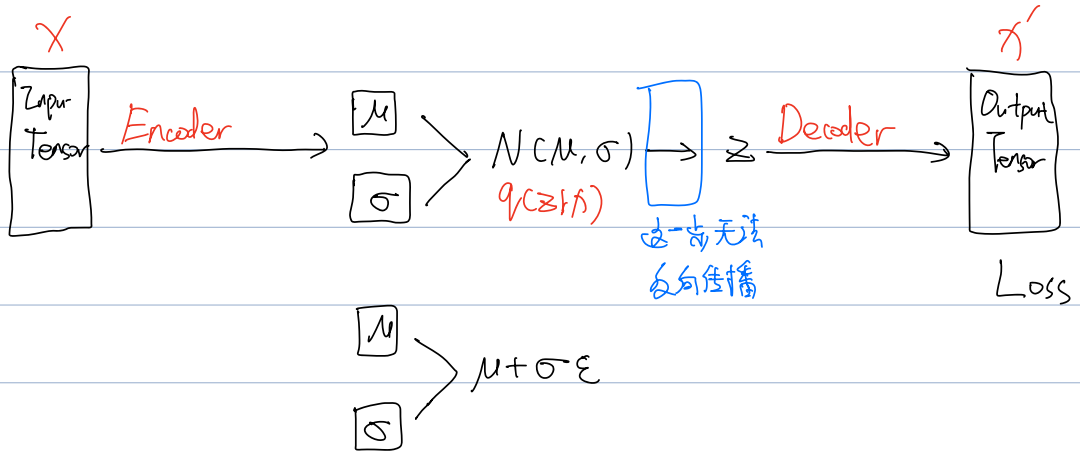
$$= E_{q(z|x)\sim z} \log\left(\frac{p(x|z)p(z)}{q(z|x)}\right)$$

$$= E_{q(z|x)}[\log p(x|z)] - KL(q(z|x)\,|\,p(z))$$

Data Consistency 项，由高斯函数性质，这个 log 值可化为 MSE

用于构建稳定潜空间，需要对于任意给定 x，生成的条件分布 $q(z|x)$ 可能靠近 $N(0,I)$

模型架构

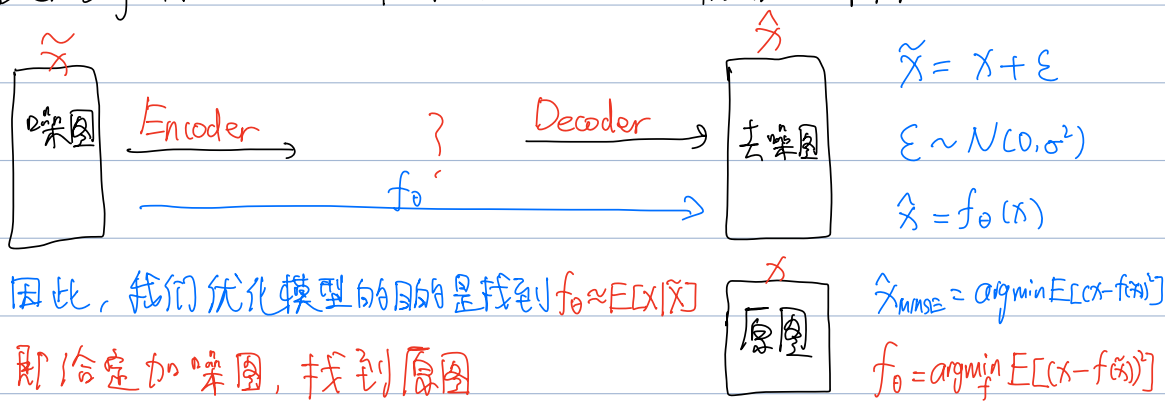


$$L = L_{KL}(N(\mu,\sigma)|N(0,1)) + L_2(x,x')$$

对高斯分布：$L_{KL} = -\frac{1}{2}(1+\log(\sigma^2)-\mu^2-\sigma^2)$

去噪自编码器

Denoising AutoEncoder 和 VAE 采用相似的模型架构：



因此，我们优化模型的目的是找到 $f_\theta \approx E[x|\tilde{x}]$

即给定加噪图，找到原图

Score：$\nabla_x \log p(x)$，一个指向密集区的梯度向量

以 $p(x) = \frac{1}{\sqrt{2\pi}} e^{-\frac{x^2}{2}}$ 为例：$\log p(x) = -\frac{1}{2}\log(2\pi) - \frac{x^2}{2}$

$\nabla_x \log p(x) = -x$（指向概率密度较大的x轴反方向）

这个公式还有个好处，这里的 $\sqrt{2\pi}$ 不见了！

这意味着 $p(x)$ 可以不是积分为1的严格概率密度函数

因此，只要跟随这个分数就能找到概率最高的区域

卷积：概率分布的叠加 $\tilde{X} = X + \varepsilon$

$$p_{\tilde{X}}(\tilde{x}) = \int p_X(x)\, p_\varepsilon(\tilde{x}-x)\, dx$$

方差越大，越模糊

Final Expression: $f_\theta(\tilde{x}) = \tilde{x} + \sigma^2 \nabla \log p_\theta(\tilde{x})$

这里很好理解，次更密集的地方有更大概率是原始图像的位置

Part II: Langevin Algorithm

由于原图存在，Loss Function 更应写成 $L = \underset{x}{\operatorname{argmin}}\, E[\|\hat{x}-x\|^2 \mid y]$ 原图

即：$\hat{x} = E[x|y]$ 最优预测等价于条件期望

$S_\theta(x_t) = x_t + \sigma^2 \nabla \log p(x_t)$

重写：$E[x|y] = y + \sigma^2 \nabla \log p(y)$，这个式子很好推导

$$\nabla \log p(y) = \frac{\nabla p(y)}{p(y)} = \frac{\nabla \int p(y|x)p(x)dx}{p(y)} = \frac{\int \nabla p(y|x)p(x)dx}{p(y)}$$

高斯加噪过程：$p(y|x) \propto \exp\left(-\frac{1}{2\sigma^2}\|y-x\|^2\right)$

$$\nabla p(y|x) = p(y|x)\cdot\left(-\frac{1}{\sigma^2}\right)(y-x)$$

$$\begin{aligned}\nabla p(y) &= \int -\frac{1}{\sigma^2}(y-x)\, p(y|x)\, p(x)\, dx \\ &= -\frac{1}{\sigma^2} y \int p(y|x)p(x)dx + \frac{1}{\sigma^2}\int p(y|x)\, x\, p(x)dx \\ &= -\frac{1}{\sigma^2} y\, p(y) + \frac{1}{\sigma^2}\int p(x|y)\, x\, p(y)\, dx \\ &= -\frac{1}{\sigma^2} y\, p(y) + \frac{1}{\sigma^2} E[x|y]\, p(y)\end{aligned}$$

$\nabla \log p(y) = -\frac{1}{\sigma^2} y + \frac{1}{\sigma^2} E[x|y]$ 推导完毕

接下来，问题就变成了寻找 $\nabla \log p(y)$，没有这个，我们就无法优化

$E_{x\sim p(x)}[\|s_\theta(x) - \nabla_x \log(p(x))\|^2]$ 不知道 $p(x)$ 无法优化

Winner: $E_{y\sim p(y),\, x\sim p(x)}[\|s_\theta(y) - x\|^2]$

对于去噪过程，$x_{t+1} = x_t + \eta \nabla \log p(x_t)$

↓ 模型拟合

$\frac{1}{\sigma^2}(s_\theta(x_t) - x_t)$

为了多样化生成：$x_{t+1} = x_t + \frac{\eta}{\sigma^2}(s_\theta(x_t) - x_t) + \sqrt{2\eta}z$ ，$z \sim (0,1)$

Score-based Model 的缺陷是无法从纯噪声图找回去

DDPM

正向马尔可夫链：$x_t = x_0 + t\beta\varepsilon$，$q(x_t|x_0) = N(x_0, t\beta)$

实际上这样不行，因为我们希望 $\lim_{t\to\infty} q(x_t|x_0) = N(0,1)$

$\Rightarrow q(x_t|x_{t-1}) = \sqrt{1-\beta}\, x_{t-1} + \beta\cdot\varepsilon$

令 $\alpha_t = 1 - \beta_t$

有：$q(x_t|x_0) = \sqrt{\bar{\alpha}_t}\, x_0 + (1-\bar{\alpha}_t)\cdot\varepsilon$ （$\bar{\alpha}_t = \prod_{i=1}^{t}(1-\beta_i)$）

逆向：$p_\theta(x_{t-1}|x_t)$，优化目标为 $-\log p_\theta(x_0)$

这个肯定无法直接优化，所以要通过数学推导来化简

定义符号：$q(x_0, x_1, x_2 \cdots x_T)|x_0) = q(x_1|x_0)\, q(x_2|x_1) \cdots q(x_T|x_{T-1})$

记作：$q(x_{1:T}|x_0) = \prod_{t=1}^{T} q(x_t|x_{t-1})$

相似的，逆向也可写成累乘：$p_\theta(x_{0:T}) = p_\theta(x_T)\prod_{t=1}^{T} p_\theta(x_{t-1}|x_t)$

$$
\begin{aligned}
-\log p_\theta(x_0) &= -\log \int p_\theta(x_{0:T})\, dx_{1:T} \\
&= -\log \int \frac{q(x_{1:T}|x_0)}{q(x_{1:T}|x_0)} p_\theta(x_{0:T})\, dx_{1:T} \\
&= -\log E_{q(x_{1:T}|x_0)}\left[\frac{p_\theta(x_{0:T})}{q(x_{1:T}|x_0)}\right] \\
&\le -E_{q(x_{1:T}|x_0)}\left[\log\left(\frac{p_\theta(x_{0:T})}{q(x_{1:T}|x_0)}\right)\right] \\
&= -E_q\left[\log(p(x_T)) + \sum_{t\ge 1}\log\frac{p_\theta(x_{t-1}|x_t)}{q(x_t|x_{t-1})}\right] \quad \text{乘以相消} \Rightarrow \log\frac{q(x_t|x_0)}{q(x_T|x_0)} \\
&= E_q\left[-\log p(x_T) - \sum_{t\ge 1}\log\frac{p_\theta(x_{t-1}|x_t)}{q(x_{t-1}|x_t,x_0)}\cdot\frac{q(x_{t-1}|x_0)}{q(x_t|x_0)} - \log\frac{p_\theta(x_0|x_1)}{q(x_1|x_0)}\right] \\
&= E_q\left[-\log\frac{p(x_T)}{q(x_T|x_0)} - \sum_{t\ge 1}\log\frac{p_\theta(x_{t-1}|x_t)}{q(x_{t-1}|x_t,x_0)} - \log p_\theta(x_0|x_1)\right] \\
&= -E_q\left[D_{KL}(q(x_T|x_0)\,\|\,p(x_T)) + \sum_{t\ge 1} D_{KL}(q(x_{t-1}|x_t,x_0)\,\|\,p_\theta(x_{t-1}|x_t)) - \log p_\theta(x_0|x_1)\right]
\end{aligned}
$$

不受模型影响。　最关键的一项　最后一次去噪，可忽略

通常取 $p(x_T)=N(0,1)$　真实后验与预测后验

优化目标：$-E_q\left[\sum_{t\ge 1} D_{KL}(q(x_{t-1}|x_t,x_0)\,\|\,p_\theta(x_{t-1}|x_t))\right]$

$$q(x_{t-1}|x_t,x_0) = \frac{q(x_t|x_{t-1},x_0)\,q(x_{t-1}|x_0)}{q(x_t|x_0)}$$ （贝叶斯）

$$q(x_t|x_{t-1},x_0) = q(x_t|x_{t-1}) = N(\sqrt{\alpha_t}x_{t-1},\ \beta_t\varepsilon)$$

$$q(x_{t-1}|x_0) = N(\sqrt{\bar\alpha_{t-1}}x_0,\ (1-\bar\alpha_{t-1})\varepsilon)$$

$$q(x_t|x_0) = N(\sqrt{\bar\alpha_t}x_0,\ (1-\bar\alpha_t)\varepsilon)$$

代入得：

$$q(x_{t-1}|x_t,x_0) \propto e^{-\frac{1}{2}\left[\frac{(x_t-\sqrt{\alpha_t}x_{t-1})^2}{\beta_t} + \frac{(x_{t-1}-\sqrt{\bar\alpha_{t-1}}x_0)^2}{(1-\bar\alpha_{t-1})} + \frac{(x_t-\sqrt{\bar\alpha_t}x_0)^2}{1-\bar\alpha_t}\right]}$$

因此，这是一个高斯分布，

$$\tilde\mu_t(x_t,x_0) = \frac{\sqrt{\alpha_{t-1}}\beta_t}{1-\bar\alpha_t}x_0 + \frac{\sqrt{\alpha_t}(1-\bar\alpha_{t-1})}{1-\bar\alpha_t}x_t$$

$$\tilde\beta_t = \frac{1-\bar\alpha_{t-1}}{1-\bar\alpha_t}\beta_t$$

我们成功把 $q(x_{t-1}|x_t,x_0)$ 算出来了，令 $p_\theta(x_{t-1}|x_t)$ 也为高斯分布，

KL 散度简化为：$\frac{1}{2\sigma_t^2}\|\tilde\mu_t - \mu_\theta\|^2$

新优化目标：$E_q\left[\sum_{t\ge 1}\frac{1}{2\sigma_t^2}\|\mu_t - \mu_\theta(x_t,t)\|^2\right]$

$$x_0 = \frac{1}{\sqrt{\alpha_t}}(x_t - \sqrt{1-\bar{\alpha}_t}\,\varepsilon)$$

$$\tilde{\mu}_t = \frac{1}{\sqrt{\alpha_t}}\left(x_t - \frac{\beta_t}{\sqrt{1-\bar{\alpha}_t}}\varepsilon\right)$$

$$\mu_\theta(x_t, t) = \frac{1}{\sqrt{\alpha_t}}\left(x_t - \frac{\beta_t}{\sqrt{1-\bar{\alpha}_t}}\varepsilon_\theta(x_t, t)\right)$$

相减均去 $\frac{1}{\sqrt{\alpha_t}}x_t$

参数相关项

待优化目标：$E_q\left[\sum_{t>1}\frac{\beta_t^2}{2\sigma_t^2\sigma_2(1-\bar{\alpha}_t)}\|\varepsilon-\varepsilon_\theta(x_t,t)\|^2\right]$

随机选一个时间，得到：

$$E_{q,t}\left[\frac{\beta_t^2}{2\sigma_t^2\sigma_2(1-\bar{\alpha}_t)}\|\varepsilon-\varepsilon_\theta(x_t,t)\|^2\right]$$

小结

VAE (2013) → Diffusion (2015) → DDPM (2020) → Score-based Diffusion Models (2021)

Denoising Score Matching (2013) → Annealed Langevin Dynamics (2019) → Score-based Diffusion Models (2021)

SDE

从 DDPM 出发：$x_i = \sqrt{1-\beta_i}\,x_{i-1} + \sqrt{\beta_i}\,\varepsilon_{i-1}$

记 $\beta_i = N\beta_i$，$\Delta t = \frac{1}{N}$

$$x_i = \sqrt{1-\frac{\beta_i}{N}}\, x_{i-1} + \sqrt{\frac{\beta_i}{N}}\, \varepsilon_{i-1}$$

$$x(t+\Delta t) = \sqrt{1-\beta(t+\Delta t)\Delta t}\, x(t) + \sqrt{\beta(t+\Delta t)\Delta t}\, \varepsilon(t)$$

泰勒展开 $x(t+\Delta t) = x(t) - \frac{1}{2}\beta(t+\Delta t)\Delta t\, x(t) + \sqrt{\beta(t)\Delta t}\, \varepsilon(t)$

$$dx = -\frac{1}{2}\beta(t)\, x(t)\, dt + \sqrt{\beta(t)}\sqrt{dt}\, \varepsilon(t)$$

$dW = \sqrt{dt}\,\varepsilon(t)$

Ito Equation $dx = -\frac{1}{2}\underbrace{\beta(t)}_{f(x,t)}\, x\, dt + \underbrace{\sqrt{\beta(t)}}_{g(t)}\, dW$

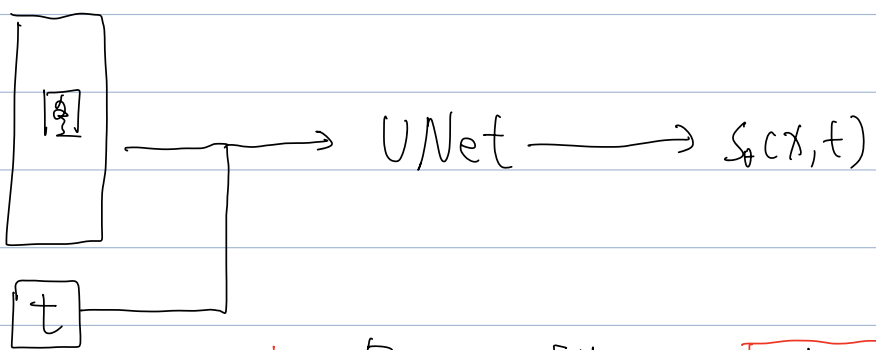
这是正向 DDPM 的连续过程，它符合 Ito Equation 形式

好消息：Ito Equation 有逆向形式，因此复原就很方便

逆向形式：$dx = [f(x,t) - g(t)^2 \nabla_x \log p_t(x)]\, dt + g(t)\, d\bar{W}$

$$dx = [-\tfrac{1}{2}\beta(t)x - \beta(t)\nabla_x \log p_t(x)]\, dt + \sqrt{\beta_t}\, d\bar{W}$$

模型架构



$$L(\theta) = E_{x_0 \sim p(x),\, t}\left[\| S_\theta(x,t) - \nabla_x \log p_t(x) \|^2\right]$$

等价于：$\nabla_x \log p_t(x_t|x_0)$

$x_t = \sqrt{1-\beta_t}\, x_0 + \beta_t \varepsilon$

$$\hookrightarrow \nabla_x \log N(\sqrt{1-\beta_t}\, x_0, \beta_t) = \frac{\sqrt{1-\beta_t}\, x_0 - x_t}{\beta_t} = -\frac{\varepsilon}{\sqrt{\beta_t}}$$

SDE: $L(\theta) = E_{x_0 \sim p(x),t}[\| S_\theta(x,t) + \frac{\varepsilon}{\sqrt{\beta_t}} \|^2]$

DDPM: $L(\theta) = E_{x_0 \sim p(x),t}[\| \varepsilon_\theta(x,t) - \varepsilon \|^2]$

$S_\theta(x,t) \propto -\varepsilon_\theta(x,t)$

Final: $dx = [-\frac{1}{2}\beta(t)x - \beta(t) S_\theta(x,t)]dt + \sqrt{\beta(t)}\, dW$

我们需要离散化来生成中间图片

$$x_{t+\Delta t} = x_t + \left[-\frac{1}{2}\beta(t)x_t - \beta(t) S_\theta(x_t,t)\right]\Delta t + \sqrt{\beta(t)}\sqrt{\Delta t}\,\varepsilon$$

DDIM

DDIM 简化了 DDPM 太慢的问题

$$p(x_s \mid x_t, x_0) = N(kx_0 + m x_t, \Delta^2 I)$$

$$\begin{aligned} x_s &= kx_0 + m x_t + \Delta\varepsilon \\ &= kx_0 + m\cdot(\sqrt{\bar\alpha_t}x_0 + \sqrt{1-\bar\alpha_t}\varepsilon') + \Delta\varepsilon \\ &= (k + m\sqrt{\bar\alpha_t})x_0 + \sqrt{m^2(1-\bar\alpha_t)^2 + \Delta^2}\,\varepsilon \quad \text{(根据方差相加)} \\ &= \sqrt{\bar\alpha_s}x_0 + \sqrt{1-\bar\alpha_s}\,\varepsilon \end{aligned}$$

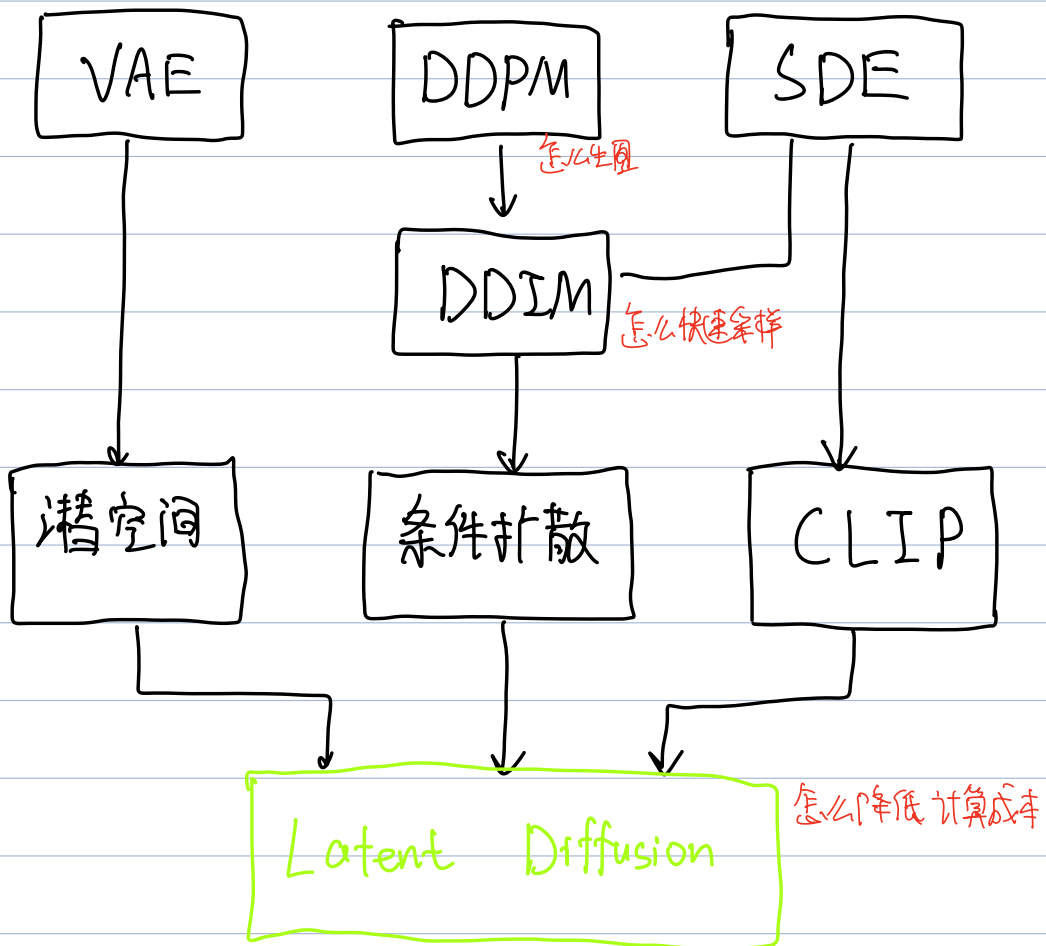
$$m = \frac{\sqrt{1-\bar\alpha_s-\Delta^2}}{\sqrt{1-\bar\alpha_t}} \qquad k = \frac{\sqrt{\bar\alpha_s} - \sqrt{1-\bar\alpha_s-\Delta^2}}{\sqrt{1-\bar\alpha_t}}\sqrt{\bar\alpha_t}$$

$$= N\left(\sqrt{\bar\alpha_s}x_0 + \frac{\sqrt{1-\bar\alpha_s-\Delta^2}}{\sqrt{1-\bar\alpha_t}}(x_t - \sqrt{\bar\alpha_t}x_0),\ \Delta^2 I\right)$$

Δ 可以取 0，也可以取 DDPM 中的 $\tilde\beta_t = \sqrt{\frac{1-\bar\alpha_{t-1}}{1-\bar\alpha_t}}\beta_t$

可以证明当 $\sigma = \tilde{\beta}_t$ 时，分布退化为DDPM形式 (令 s=t-1, k=t)

☆：DDIM 不改变训练过程，它和DDPM使用同一个 ϵ_θ，但 DDIM 把 ϵ_θ 代入跳步公式加快采样



Latent Diffusion

把加噪/去噪转移到潜空间，在保留质量的前提下大大加速图像生成

Diffusion 模型在原始图像空间下训练与推理极其消耗时间与算力（训练一个顶尖DM需要成百上千GPU时）
（carbon footprint 也很大，神了）

① 它利用 VAE 和 GAN 的损失函数思想训练了一个 Encoder 将图像编码到潜空间

② 在潜空间复用 DDPM

③ Cross-Attn 注入文本等条件，To 将多模态输入编码到 Intermediate-Layer

IP-Adapter

直接由文本生图效果不尽如人意，IP-Adapter 解决了多模态输入

Generative Modeling via Drifting

Drifting Model 把迭代过程放到训练阶段，试图一次性在 Pixel Space 训练一个从 $N(0,I)$ 到训练集图像空间 p 的分布映射

① 从 $N(0,I)$ 采样：$\varepsilon_i \sim N(0,I)$

② 经过变换器：$x_i = f_\theta(\varepsilon_i)$

③ $q_\theta = f_{\theta\#}N(0,I)$，$q_\theta$ 为输出分布

④ p：从训练集中采样一部分图片

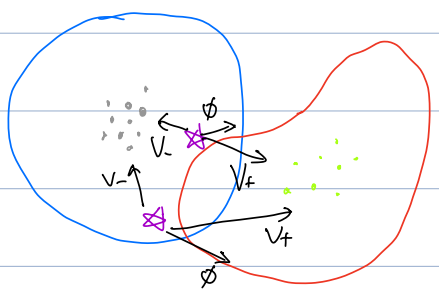
⑤ 计算2个样本加权中心，得到偏移向量 $V_{p,q}(x_i)$

⑥ $\hat{x}_i = x_i + V_{p,q}(x_i)$，计算 $L = \frac{1}{B}\sum_{i=1}^{B} \|x_i - sg(\hat{x}_i)\|^2$

⑦ 反向传播，更新 θ

由于图像像素无法衡量语义，可以用一个 Encoder 把 x 编码到语义空间（用 ϕ 表示）

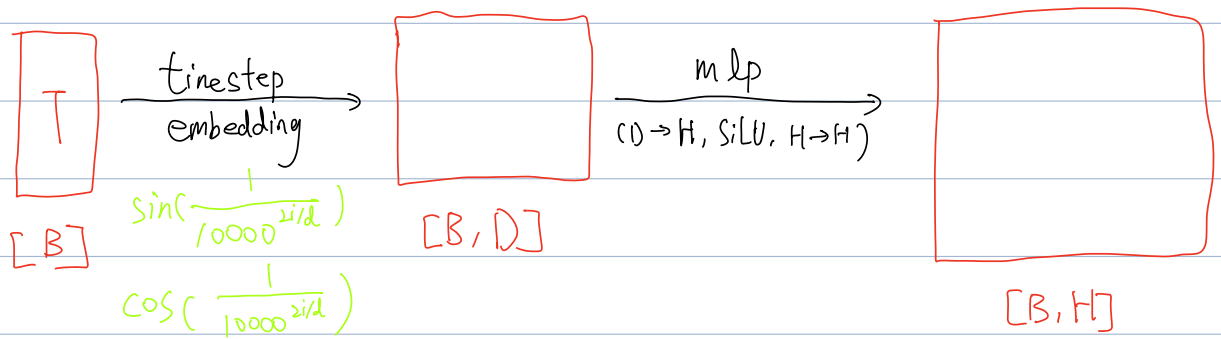
loss：$E\left[\|\phi(x) - \text{stopgrad}(\phi(x) + V(\phi(x)))\|^2\right]$



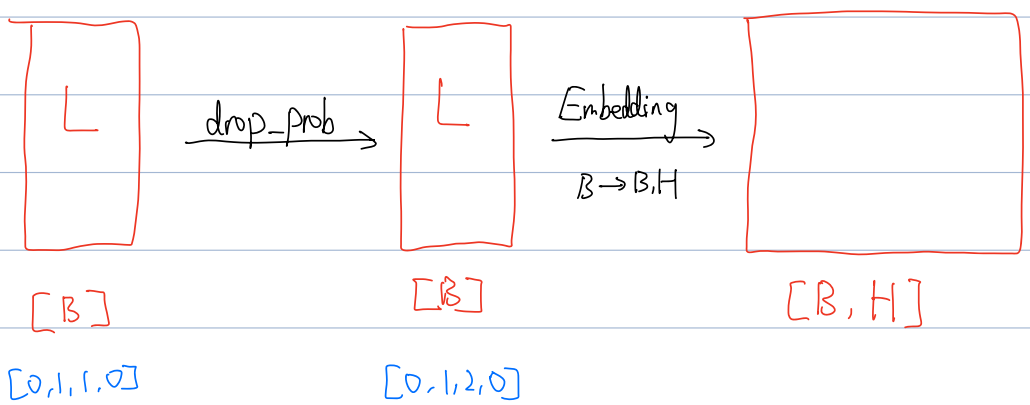
DiT

模型架构

TimeEmbedding



LabelEmbedder 以 0:猫，1:狗，2:无类别 为例



DiT Block（本质 Transformer，多头注意力 + LayerNorm + ResNet + FFN）

